

CDP: 13200175

EXAMEN PARCIAL DESARROLLO DE SISTEMAS MÓVILES

ANÁLISIS DE ARQUITECTURA Y CONCURRENCIA.

1. El patrón Repositorio como SSOT, se considera la única fuente de verdad porque es el componente central que se encarga de orquestar el flujo de datos, de esta manera se asegura que la UI siempre reciba información consistente. También actúa como fachada porque oculta la complejidad de la lógica de datos.
 - Implementa el cache-then-network, primero consulta la cache local (room) para emitir datos inmediatamente a la UI y así evitar pantallas vacías. A la vez dispara una petición asincrónica al retrofit, al recibir con éxito, actualiza la BD local mediante la operación UPSERT, provocando que Room notifique automáticamente a la UI mediante el flow.
2. CORROUTINAS Y STRUCTURED CONCURRENCY.
 - El ViewModelsScope es una corrutina vinculado al ciclo de vida del ViewModel. Es fundamental para la gestión de memoria porque cancela automáticamente todas las corrutinas activas cuando el ViewModel se destruye.
 - Si el usuario cierra la pantalla antes de recibir una respuesta de red, el scope se cancela y la petición se interrumpe, esto evita los memory leaks que son las fugas de memoria.
3. Flujos Reactivos vs Hot.
 - El flow es un flujo secuencial que no consume recursos ni emite datos hasta que existe un recolector activo (collect).
 - El stateflow es un flujo diseñado para el manejo de estados de estado de la UI que siempre almacena el último valor.
 - Se prefiere stateflow para la interfaz de usuario porque garantiza que la UI siempre tenga un estado disponible, incluso tras cambios de configuración como una rotación de pantalla.
4. INYECCIÓN DE DEPENDENCIAS (DI) MANUAL.
LA CLASE APPLICATION ACTUA COMO CONTENEDOR GLOBAL DONDE SE INSTANCIAN LAS DEPENDENCIAS QUE DEBEN SOBREVIVIR DURANTE LA EJECUCIÓN DE LA APP
by lazy se utiliza para la inicialización perezosa, es decir que el objeto no se crea al momento del arranque de la app, sino solo la primera vez que es requerido en algún componente

II. COMPLEJAR.

1. El flujo de datos que emite AUTOMATICAMENTE cada vez que la TABLA DE ROOM CAMBIA SE IMPLEMENTA USANDO `flow<list>`
2. El OPERADOR DISPATCHER, PERMITE EJECUTAR TAREAS PESADAS FUERA DEL MAIN THREAD, ESPECIFICAMENTE OPTIMIZANDO PARA TAREAS DE CPO COMO EL PARSEO NATIVO DE DATOS.
3. PARA QUE UN SERVICIO EN PRIMER PLANO DE UBICACION SEA VALIDO EN ANDROID 10+, DEBE DECLARARSE EN EL MANIFESTO CON EL TIPO `LOCATION`.
4. LA FUNCION
5. EL COMPONENTE QUE PERMITE TRANSFORMAR UNA API BASADA EN CALLBACKS ANTIGUOS ES UNA FUNCION SUSPENDIBLE NO DEBERIA LLAMAR `CALLBACK(fw)`.
6. EL ARCHIVO DE METADATOS QUE DESCRIBE LA ESTRUCTURA DE LA APP AL SISTEMA OPERATIVO SE LLAMA EXACTAMENTE `AndroidManifest.xml`
7. EN LA ARQUITECTURA MOSP, LA CAPA QUE ACTUA COMO PUENTE STANDARD ENTRE EL FRAMEWORK Y LOS CONTROLADORES SE DENOMINA HAL
8. EL OPERADOR DE FLOW QUE PERMITE COMBINAR MULTIPLES FUENTES (COMO GPS Y SENSORES) PARA EMITIR UN ESTADO UNIFICADO ES, EL `COMBINE`.

III. DESARROLLO DE CODIGO.

1. REPOSITORIO Y DI

// Repositorio

```
class UsuarioRepository (private
```

```
    val usuarioDao: UsuarioDao) {
```

```
    fun getUsers() = usuarioDao.getAll()
}
```

// Application

```
class MyApp: Application() {
```

```
    private val database by lazy {
```

```
        AppDatabase.getInstance(this)
    }
```

// Inyeccion MANUAL del Repositorio.

```
    val usuarioRepository by lazy {
```

```
        UsuarioRepository(database.usuarioDao())
    }
```

```
}
```

2. Lógica Dinámica de Identidad:

```
a) DATA CLASS USUARIO (  
    VAL NOMBRE: STRING,  
    VAL APELLIDOS: STRING,  
    VAL EDAD: INT?
```

```
b) fun MAIN () {  
    VAL miUsuario =  
        USUARIO ("ERICK", "GUISPE", 29)
```

```
c)  
    VAL nombreLen = miUsuario.nombre?.length?: 0  
    VAL apellidoLen = miUsuario.apellidos?.length?: 0  
    VAL PUNTAJE TOTAL = nombreLen + apellidoLen
```

```
d)  
    PRINTln ("USUARIO: $  
        { miUsuario.nombre } $  
        { miUsuario.apellidos }, tu puntaje de  
        Identidad es: $PUNTAJE TOTAL")  
}
```